

Optimal Resizable Arrays — 论文 Review

Tarjan & Zwick (2024): Optimal Resizable Arrays
SIAM Journal on Computing, 53(5), 1354–1380

周希 李宇轩 吴贤文

1 引言：一个“看起来简单”的基础问题

1.1 可调整数组问题

可调整数组（resizable array）是计算机科学中最基础的数据结构之一。它需要在支持按下标 $O(1)$ 随机访问的前提下，允许在数组末尾动态地增加（Grow）或删除（Shrink）元素。几乎所有高级编程语言的标准库都提供了这一结构——C++ 的 `std::vector`，Java 的 `ArrayList`，Python 的 `list`。

经典教科书的解决方案是**倍增策略**（doubling / geometric expansion）：数组满时分配一个两倍大小的新数组并拷贝全部元素；当数组使用率低于 $1/4$ 时，分配一半大小的新数组。均摊分析表明每次操作的均摊代价为 $O(1)$ ，但空间利用率在最坏情况下仅约 50%——即需要 $\Theta(N)$ 的额外空间。

1.2 从 $\Theta(N)$ 到 $O(\sqrt{N})$ ：一个 25 年的突破

1996 年，Sitariski 提出了 HAT（Hashed Array Tree），首次将额外空间降至 $O(\sqrt{N})$ 。1999 年，Brodnik 等人进一步给出了只需 $N + O(\sqrt{N})$ 空间且支持 $O(1)$ 最坏情况操作的结构，并证明了 $\Omega(\sqrt{N})$ 的额外空间下界。这一结果看似为可调整数组问题画上了句号——但事实并非如此。

1.3 Tarjan & Zwick (2024) 的核心贡献

Tarjan 和 Zwick 的这篇论文做出了一个关键的概念突破：将“存储空间”与“临时调整空间”分离。他们发现，Brodnik 等人的 $\Omega(\sqrt{N})$ 下界是将两者混为一谈得到的。一旦区分平时存储数组所需的空间 $s(N)$ 和调整大小时短暂使用的临时空间 $t(N)$ ，就可以“绕过” $\sqrt{N}$ 墙——平时存得更紧凑，仅在调整瞬间多用一点空间。

论文的核心结果是建立了一个以整数 $r \geq 2$ 为参数的完整权衡谱系：

- $r = 2$ ：退化回经典结果——存储额外 $O(\sqrt{N})$ ，临时额外 $O(\sqrt{N})$ ，均摊 $O(1)$ 。

表 1: 核心结果: r 参数化的最优权衡谱系

指标	上界 (构造性)	下界 (不可能更好)
稳定存储额外空间 $s(N)$	$O(N^{1/r})$	$\Omega(N^{1/r})$
增长时临时额外空间 $t(N)$	$O(N^{1-1/r})$	$\Omega(N^{1-1/r})$
增长/收缩均摊时间	$O(r)$	$\Omega(r)$
随机访问	$O(1)$ 最坏情况	—

- $r = 3$: 存储额外 $O(N^{1/3})$, 临时额外 $O(N^{2/3})$, 均摊 $O(1)$ 。
- $r = \log N$: 存储额外 $O(\log N)$, 但操作时间升至 $O(\log N)$ 。

上下界在所有 r 处匹配, 说明这个三维 trade-off (存储–临时–时间) 已被完全解决。

2 问题形式化与计算模型

2.1 $(s(N), t(N))$ -实现

论文的核心形式化创新是定义 2.1 中的 $(s(N), t(N))$ -实现:

- **稳定状态**: 存储长度为 N 的数组时, 使用空间 $\leq N + s(N)$ 。
- **调整期间**: 对长度为 N 的数组执行 Grow/Shrink 时, 使用空间 $\leq N + t(N)$ 。

$s(N)$ 和 $t(N)$ 均为非递减函数。这个看似简单的区分是整个论文理论大厦的基石——它将“平时”和“调整时”的空间约束分开衡量, 使得精细的 trade-off 分析成为可能。

2.2 块模型

数据结构由若干固定长度的连续内存块 (block) 组成:

- **数据块**: 存放实际元素, 每元素占一个 word。
- **索引块**: 存放指向其他块的指针, 每指针占一个 word。
- **辅助块**: 存放元数据 (如长度、容量等)。

除主索引块外, 每个可访问的块必须被某个索引块中的指针指向——这一“可寻址性”假设是后续所有下界证明的关键。

3 前人工作：从倍增数组到 Brodnik 结构

3.1 几何扩展/收缩与均摊分析

最经典的 α -几何扩展策略选择参数 $\alpha > 0$ ：数组满时分配大小为 $(1 + \alpha)N$ 的新数组并拷贝 N 个元素；使用率低于 $1/(1 + \alpha)^2$ 时收缩为 $N/(1 + \alpha)$ 。

在阅读过程中，我们独立推导了该策略的势能函数。对于增长因子 $\beta = 1 + \alpha$ ，势能函数的标准形式为 $\Phi = \frac{1+\alpha}{\alpha} \cdot n - \frac{1}{\alpha} \cdot M$ ，其中 n 为当前元素数， M 为当前容量。验证其正确性：扩容前 $n = M$ 时 $\Phi = M$ ，恰好等于需拷贝的 M 个元素的代价；扩容后 $M = (1 + \alpha)n$ 时 $\Phi = 0$ ，势能释放完毕。我们的初步推导中曾因势能函数系数偏差而得到错误结果——多了一个 $(1 + \alpha)$ 因子——这源于对扩容实际代价的误解：扩容时拷贝的是 N 个旧元素（代价 N ），而非 $(1 + \alpha)N$ 。经过与 CLRS 经典结果（ $\alpha = 1$ 时 $\Phi = 2n - M$ ）的交叉验证，我们修正了这一错误。这种从偏差到修正的过程，让我们对势能法的理解从公式记忆上升到了对“势能应恰好支付昂贵操作代价”这一核心原则的把握。

由势能函数可得每次插入的均摊代价为 $2 + 1/\alpha = O(1)$ 。 α 越大，均摊时间越省（趋近于 2），但空间利用率 $1/(1 + \alpha)$ 越低——不存在同时最优化时间和空间的 α 。

当增长和收缩同时存在时，若收缩阈值设置不当（如使用率低于 $1/2$ 就收缩），对抗性操作序列可能导致“抖动”（thrashing）：通过不断在临界点来回跨越，每次触发 $\Theta(N)$ 的拷贝代价，均摊退化至 $\Omega(N)$ 。标准的解决方法是将收缩阈值设为 $1/4$ 而非 $1/2$ ，使增长阈值与收缩阈值之间相差因子 4，保证任何跨越都需要 $\Omega(N)$ 次廉价操作。我们注意到，这个“滞后”（hysteresis）设计反映了动态数据结构中一个更普遍的原则：**做决策的边界必须比决策产生的变化更宽**。这一原则在操作系统页面置换、缓存失效策略等领域同样成立——本质上都是用“对变化的适度不敏感”换取“系统行为的稳定”。

论文指出，可通过**后台重建**（background rebuilding）将倍增数组的均摊 $O(1)$ 去均摊化为最坏情况 $O(1)$ 。其核心并不涉及多线程：只需同时保留新旧两个数组，每次正常的 Grow/Shrink 操作顺手从旧数组复制常数个元素到新数组。访问时根据下标与复制进度 p 的关系做三支判断（已复制区 / 未复制区 / 扩容后新增区），仍保持 $O(1)$ 。因为新数组从半满到再次装满需要 $\Theta(N)$ 次操作，恰好足够完成旧数组全部元素的复制——原本一次 $\Theta(N)$ 的峰值代价被均匀分散到 $\Theta(N)$ 次操作中。收缩的方向类似：从数组前部开始复制，尾部被删除的元素无需搬运。

3.2 HAT (Hashed Array Tree)

Sitarski (1996) 的 HAT 是一种两级结构。维护参数 $B = 2^p$ （满足 $\sqrt{N} \leq B < 4\sqrt{N}$ ），包含一个大小为 B 的顶层索引块和若干大小为 B 的数据块。第 i 个元素位于第 $\lfloor i/B \rfloor$ 个数据块的第 $i \bmod B$ 个位置——通过位运算 $i \gg p$ 和 $i \& (B - 1)$ 即可在硬件层面瞬时完成寻址。

HAT 的总空间为 $N + 3B + 2 = N + O(\sqrt{N})$ 。我们对这三项 B 级开销做了逐项

确认：(1) 索引块本身 B 个指针；(2) 当前正在填充的数据块（可能未满，最坏浪费 $B-1$ ）；(3) 预分配的一个备用空数据块（大小为 B ，用于增长时无需立即向 OS 申请）。常数 $+2$ 为记录 size 和 B 的元数据。第 3 个 B （备用块）是空间换时间的典型体现：多投入 B 的空间，换取最坏情况 $O(1)$ 的增长操作。

当 $N = B^2$ 时触发全局重建（ B 加倍）， $N = B^2/16$ 时触发反向重建（ B 减半）。重建后 $N = B^2/4$ ——无论是扩建还是收缩，都恢复到“四分之一满”的中间状态。这意味着下次扩建需要净增 $3N/4$ 个元素，下次收缩需要净减 $3N/16$ 个元素——无论如何，两次重建之间至少有 $\Omega(N)$ 次操作，重建的 $O(N)$ 成本被充分分摊。

3.3 Brodnik 等人的变长数据块结构

Brodnik 等人 (1999) 的结构不再使用统一大小的数据块，而是采用依次增大的数据块序列。算数级数版本使用容量 $1, 2, 3, \dots, k$ 的数据块，总容量 $k(k+1)/2 \approx N$ ，故 $k = \Theta(\sqrt{N})$ ，最后一个块和索引块的开销均为 $O(\sqrt{N})$ 。由于不需要改变块大小，无需全局重建，操作可达最坏情况 $O(1)$ 。几何级数版本（超级块）将块大小改为 2 的幂次——第 q 个超级块包含 $2^{\lfloor q/2 \rfloor}$ 个大小为 $2^{\lceil q/2 \rceil}$ 的数据块，总容量恰好 2^q ——用位运算和前置零计数指令取代了开平方根运算。

我们思考了一个逆向问题：能否反过来使用“变长索引块 + 定长数据块”？答案是否定的。当索引块长到极限时，为了继续插入必须改变底层数据块的基准大小，这会导致所有旧数据重新搬运，不可避免地触发 $\Theta(N)$ 的大卡顿——使去均摊化的努力前功尽弃。这反过来解释了为什么 Brodnik 选择让数据块变长而非索引块变长的设计是内在必然的。我们还注意到，算数级数与几何级数两种方案代表了两种不同的寻址哲学：前者利用等差数列求和公式（需开平方根），后者利用 2 的幂次和位运算——后者在现代硬件上明显更友好，体现了理论设计中对实现效率的考量。

4 空间下界：从 $\sqrt{N}$ 到 $s(N) \cdot t(N) \geq N$

4.1 $\sqrt{N}$ 下界的回顾与鲁棒性

定理 4.1 证明任何可调整数组实现（即使只做 Grow + Access）也必须在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间。证明采用精巧的二分讨论：设 N 次 Grow 后使用了 k 个内存块——

- **情形一** ($k \geq \sqrt{N}$)：每个块需要一个指针 $\rightarrow$ 指针开销 $\geq k \geq \sqrt{N} \rightarrow$ 空间 $\geq N + \sqrt{N}$ 。
- **情形二** ($k < \sqrt{N}$)：总容量 $\geq N$ ，但块数少于 $\sqrt{N}$ ，由平均值原理最大块容量 $\ell \geq N/k > \sqrt{N}$ 。关键的一步是考察该大块刚被分配的时刻：此时块内尚无元素，但已占据 ℓ 空间，原有的 N' 个元素仍存于旧块中 $\rightarrow$ 瞬时空间 $\geq N' + \ell > N' + \sqrt{N'}$ 。

无论设计策略偏向“多小块”还是“少大块”，都无法逃脱这个二分夹击。

我们进一步考察了该下界在指针可压缩的硬件上的鲁棒性。若物理内存仅 2^{cw} ($0 < c < 1$)，指针只需 cw 位即可打包存储。情形一的阈值调整为 $k \geq (1/\sqrt{c}) \cdot \sqrt{N}$ ，指针开销 $\geq \sqrt{c} \cdot \sqrt{N}$ 。由于 c 为常数， $\Omega(\sqrt{c} \cdot \sqrt{N}) = \Omega(\sqrt{N})$ ，下界不受影响。这说明定理 4.1 不依赖于“指针恰好占一个 word”的特定硬件假设。

4.2 定理 4.2：乘积下界的直觉

定理 4.2 将定理 4.1 推广为更一般的形式：任意 $(s(N), t(N))$ -实现必须满足 $s(N) \cdot t(N) \geq N$ 。

证明直觉如下：若平时额外空间 $\leq s(N)$ ，则数据块数量 $\leq s(N)$ （每块需一个指针）。由平均值原理，存在某块 B 容量 $\geq N/s(N)$ 。设 B 在第 $N' \leq N$ 次 Grow 时分配——刚分配时 B 为空，原有 N' 个元素仍在旧块中，故该时刻的临时额外空间 $\geq N/s(N)$ 。由 t 的单调性， $t(N) \geq t(N') \geq N/s(N)$ ，即 $s(N) \cdot t(N) \geq N$ 。

我们认为这个乘积形式比 Brodnik 等人原始的加和下界 $s(N) + t(N) = \Omega(\sqrt{N})$ 更强且更优美，一个直接的好处是它自然地导出了 r 参数化的 **trade-off**：设 $s(N) = N^{1/r}$ ，则 $t(N) = N^{1-1/r}$ 。 $r = \log_N(1/s(N))$ 这一参数化直接从乘积下界中涌现——一个好的下界形式本身就能暗示紧的上界结构。

4.3 推论 4.3

代入 $s(N) = O(N^{1/r})$ ，立得 $t(N) = \Omega(N^{1-1/r})$ 。“如果想平时存得特别紧凑，调整时就必然付出巨大的临时空间代价”。这里的“偶尔”（occasionally）是存在性结论——某些 N 处的 Grow 操作无法避免这个临时空间峰值，而非每次 Grow 都触发。这个区分让我们意识到：在设计系统时，resize 时的瞬态内存峰值可能才是决定方案能否实装的真正瓶颈，而稳态指标并不能说明全部问题。

5 上界构造：从 $r = 3$ 到一般 r 的层次化结构

5.1 $r = 3$ ：两层块的简单特例

第 5 节给出 $(O(N^{1/3}), O(N^{2/3}))$ -实现，是理解整个上界构造的最佳入口。

核心矛盾：若直接用大小为 $N^{2/3}$ 的大数据块，最后一个块可能几乎为空，造成 $\Theta(N^{2/3})$ 的稳定空间浪费，无法达到 $O(N^{1/3})$ 的额外空间目标。

解决思路：不让大块部分填充。所有大块保持全满，数组末尾不足一个大块的部分用 HAT 存储——该 HAT 管理的元素数 $x < N^{2/3}$ ，额外空间为 $O(\sqrt{x}) \leq O(\sqrt{N^{2/3}}) = O(N^{1/3})$ ，恰好满足目标。这种“用更精细结构处理尾部”的递归思路非常自然。

非递归实现：取 $B = \Theta(N^{1/3})$ ，大块大小为 B^2 ，小块大小为 B 。大块始终全满；小块至多 $2B$ 个（最多一个半满、一个全空）。两个索引块的开销、小块未全满/全空的浪费均为 $O(B) = O(N^{1/3})$ 。当 $2B$ 个小块全满时，取其中 B 个合并为一个新大块—— $B \times B = B^2$ ，恰好填满，维持“大块永远满”的不变式。合并拷贝 B^2 个元素，

但两次合并之间至少发生 B^2 次 Grow，均摊 $O(1)$ 。缩减时若无非空小块，将一个大块拆为 B 个小块，分析对称。

在精读第 5 节时，我们通过几个自检问题验证了对结构细节的理解：(1) 若让最后大块部分填充，最坏浪费 $\Theta(N^{2/3})$ ，超出 $O(N^{1/3})$ 目标；(2) 至多 $2B$ 个小块的冗余缓冲，避免一次 Grow 就触发合并、紧接着一次 Shrink 又触发拆分的临界点抖动；(3) 合并拷贝 B^2 个元素，触发间隔 $\Theta(B^2)$ 次操作，故均摊 $O(1)$ ；(4) 令 $s(N) = O(N^{1/3})$ ，定理 4.2 给出 $t(N) = \Omega(N^{2/3})$ ，本节的 $O(N^{2/3})$ 构造恰好达到下界——该权衡渐近最优。这种自检式阅读帮助我们将论文的每一个“为什么”落实为可验证的判断。

5.2 一般 r ：多层块与 Credit 均摊分析

第 6 节推广为任意 $r \geq 2$ 的多层结构，给出 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现。

冗余 B 进制计数器：令 $B = \Theta(N^{1/r})$ ，使用大小为 $B, B^2, \dots, B^{r-1}$ 的块。每层允许至多 $2B$ 个块（冗余因子 2，防止抖动）。Grow 触发“进位”—— B 个满的 B^i 块合并为 1 个 B^{i+1} 块；Shrink 触发“借位”—— B^{i+1} 块拆为 B 个 B^i 块。最高层新块大小为 $B^{r-1} = N^{1-1/r}$ ，故临时额外空间为 $O(N^{1-1/r})$ 。

Credit 均摊账本：这是第 6 节分析中最精巧的部分。我们的理解是：credit 不是代码中真实保存的数值，而是均摊分析中的虚拟账本——把 credit 总额视为势能 Φ ，若某步实际成本为 c 同时账户等额减少，则其均摊成本 $c + \Delta\Phi = 0$ ，该操作的代价被此前预收的 credit 完全抵消。

具体来说，每个新元素进入第 1 层时被预收 r 个 credit：1 个支付当前写入，其余 $r-1$ 个随元素保存。元素每从第 i 层升至第 $i+1$ 层，所持 credit 从 $r-i$ 降为 $r-i-1$ ，释放 1 个恰好支付这一次复制。这相当于**每个元素为自己的未来搬运预付费用**——这一视角比单纯记忆 $r-i$ 公式直观得多。

拆分则相反：元素降层后未来可提升次数增加，需要更多而非更少 credit，因此不能靠元素自身支付。论文的方案是每次 Shrink 向每层公共账户存入 3 credit。两次对第 k 层的拆分之间至少发生 B^k 次 Shrink，积累至少 $3 \cdot B^k$ credit——恰好足够支付复制 B^k 个元素（ $1 \cdot B^k$ ）和补发降层元素所需的 credit（至多 $2 \cdot B^k$ ）。以 $B=4, k=3$ 为例：拆分一个 64 元素的块需复制 64 + 补发 80 = 144 $\leq 3 \times 64 = 192$ ，账户预算充足。

需要特别注意：第 6 节的 $O(r)$ 是均摊时间，单次 Grow 的实际最坏代价可能是 $\Theta(B^k)$ （触发级联合并一路进位到顶层）。论文的 credit 机制不是让这种昂贵操作变便宜，而是从数学上证明它的代价可由此前的廉价操作预支。

5.3 消去 r 因子：第 7 节的数据结构变换

第 6 节的稳定额外空间为 $O(rN^{1/r})$ 。当 r 为常数时 r 因子无关紧要，但当 r 随 N 缓慢增长时（如 $r = \log N$ ），这个 r 因子不可忽略。第 7 节通过一个通用的数据结构变换将其降至 $O(N^{1/r})$ 。

我们仔细推演了定理 7.3 的参数选择逻辑。作者首先对第 6 节取参数 $q = 2r$ （而非 r ）——之所以取 $2r$ ，是因为第 6 节对任意 q 给出的稳定界是 $O(qN^{1/q})$ ，直接令 $q = r$ 得到的 r 因子无法在此后的变换中被消除。取 $2r$ 后指数从 $1/r$ 变为 $1/(2r)$ ， $N^{1/(2r)}$ 显著更小，使得项 $rN^{1/(2r)}$ 可以被后续的 $N^{1/r}$ 吸收。然后令切分上限 $\tau(N) = N^{1-1/r}$ ，将可能很大的块切成多个大小为 $\tau(N)$ 的小块，新稳定项变为 $O(rN^{1/(2r)} + N^{1/r})$ 。当 $r \leq \frac{1}{2} \frac{\log N}{\log \log N}$ 时，由 $2r \log r \leq \log N$ 推出 $rN^{1/(2r)} \leq N^{1/r}$ ，第一项被第二项吸收——最终稳定额外空间 $= O(N^{1/r})$ 。

这里有一个易混淆的地方：引理 7.2 中的 $t(N)$ 是主动选择的切分上限（我们更愿意记为 $\tau(N)$ 以作区分），而非沿用第 6 节原有的临时空间界。本节消去的仅是稳定空间中的 r 因子，Grow/Shrink 的 $O(r)$ 均摊时间保持不变——第 9 节将说明这个时间也是紧的。

5.4 代码验证

为了验证对论文算法和均摊分析的理解，我们对核心数据结构进行了 Python 实现。实现采用了“扁平存储 + 块元数据 + 代价跟踪”的架构：实际元素存储在 Python 列表中（保证 $O(1)$ 访问正确性），块结构通过整数计数器（`partial` 记录部分块元素数，`f[i]` 记录第 i 层满块数）跟踪，合并/拆分操作不实际移动元素而是累加拷贝代价计数器——这使得我们可以精确测量操作代价而不影响功能正确性。

实现的模块包括： α -几何增长动态数组 (§3.1)、HAT 数据结构 (§3.2, $r = 2$)、通用 r 参数化实现 (§5-7)、增长游戏模拟 (§8-9)，以及综合演示与对比程序。全部正确性测试通过。在 $N = 10000$ 下得到的经验数据如下：

表 2: $N = 10000$ 下不同 r 的经验均摊代价

r	B	经验均摊代价	理论均摊	合并次数
2	128	1.55	$O(2)$	0
3	32	2.48	$O(3)$	28
5	8	3.58	$O(5)$	246
10	4	5.64	$O(10)$	1239

经验均摊代价随 r 增长的趋势与论文 $O(r)$ 的理论预测定性一致，验证了我们对均摊分析的理解。

需要诚实地指出，这是一个**算法教学实现**而非严格的性能复现：我们使用扁平列表绕过了 r 级索引的真实访问开销，代价是计数器模拟而非真实 `malloc/memcpy`，未测量缓存局部性等真实性能指标。若要做严格复现，需用 C/Rust 重写并真实管理内存。

6 增长游戏与时间下界

6.1 增长游戏的抽象逻辑

第 8 节引入增长游戏 (Growth Game)，将标准数据结构的 Grow 操作抽象为一个可精确求解的单人组合游戏。这是全文技术含量最高的部分。

(N, k, ℓ) -增长游戏： k 个子数组，空子数组必须形成前缀（这保证了状态的规范性，避免同一逻辑布局因块排列不同而重复计数）；第一个非空子数组至多有 ℓ 个空位，其余全满。三种操作：有空位 $\rightarrow$ 写入（代价 1，无选择）；全满但有空子数组 $\rightarrow$ 新分配（代价 1，无选择）；全满 $\rightarrow$ 必选某个 i 合并前 i 个子数组（代价 $= 1 + \sum$ 被合并元素总数）。

这个游戏比真实数据结构**更宽松**：它预知最终 N ，即使当前规模很小也可使用按最终 N 计算的额外空间，且无限制 Grow 的临时空间。宽松意味着游戏的最优代价 $\leq$ 真实实现的最优代价。因此，若连宽松游戏都至少需要代价 L ，真实实现必 $\geq L$ ——这保证了游戏下界的有效性。这个“削弱模型以证下界”的技巧非常经典。

6.2 归约、精确解与二项式系数

引理 8.2 将 $\ell > 0$ 归约到 $\ell = 0$ ：每 $\ell + 1$ 个元素视为一个超级元素。我们注意到精确等式要求 $\ell + 1 \mid N$ ——不整除时，最后存在不完整批次，影响有限规模的精确值，但不改变渐近结论。这个细节在阅读中容易被忽略，但它揭示了精确解与渐近下界之间的微妙关系：论文追求的是渐近最优性，这给边界情况留出了容忍空间。

定理 8.6 ($\ell = 0$ 的精确解)：当 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$ 时，

$$C_{N,k} = (N+1) \cdot n - \binom{n+k}{k+1}$$

其中 $(N+1) \cdot n$ 是“不计任何节约”的总代价（每插入一个元素就记录一次合并开销）， $\binom{n+k}{k+1}$ 是“最优策略的节约”——通过精心选择最终状态（哪些子数组保持满、哪些为空）避免的合并代价。均摊下界为 $A_{N,k} \geq \frac{k}{k+1}(n-1) \geq \frac{1}{2}(n-1)$ 。

二项式系数在游戏中扮演的不是装饰性角色，而是刻画了**各层块容量的平衡阈值**：最优状态中第 i 层块的大小必须落在 $[\binom{n+i-2}{i}, \binom{n+i-1}{i}]$ 区间内。直观上，低层块反复合并上升到高层时，各层的元素数分布自然满足 Pascal 恒等式与 hockey-stick 恒等式所描述的递推关系。交换论证进一步证明：超出此区间的分配必非最优——某个块过小必然意味着另一个块过大，将一个元素从过饱和层移至不足层，边际复制代价的差值严格为负。

这种“把操作代价编码进组合结构”的方法令我们印象深刻。增长游戏剥离了数据结构的实现细节——不再关心指针、索引块、内存分配——只保留最核心的约束（块容量、空位限制、合并代价），结果却发现最优策略完全由二项式系数的组合性质刻画。**有时候，离开一个问题才能更好地解决它。**

6.3 定理 9.1: 均摊代价 $\Omega(r)$

第 9 节将增长游戏映射回数据结构: 令 $k \approx r \cdot N^{1/r}$ (存储空间 $O(N^{1/r})$ 约束下最大可能的块数), 代入推论 8.13 的均摊下界, 取 $n = \varepsilon r$, 利用二项式上界 $\binom{n}{k} \leq (en/k)^k$, 得均摊代价 $\geq \Omega(r)$ 。与第 6-7 节构造的 $O(r)$ 上界完全匹配。

这里有一个贯穿全文的关键事实值得强调: $r = 2$ 是去均摊化的临界点。当 $r = 2$ 时, $N^{1/r} = N^{1-1/r} = N^{1/2}$ ——稳定额外空间与临时额外空间的量级相同, 新旧结构可以跨越多次操作持续共存, 后台重建可行。但当 $r > 2$ 时, $N^{1-1/r} \gg N^{1/r}$, 临时空间若跨越多次操作就会变成“平时存储空间”, 违反 $O(N^{1/r})$ 的紧空间约束。因此, 后台重建技术在此失效, 论文对 $r > 2$ 只给出 $O(r)$ 均摊而非最坏情况时间——这不是证明的缺陷, 而是空间约束本身导致的必然结果。

7 总结与评价

7.1 论文的核心贡献链

乘积下界 $s(N) \cdot t(N) \geq N$ (§4)
 $\downarrow$ 暗示 r 参数化
 上界构造: 基数 B 计数器式的层次化块结构 (§5-6)
 $\downarrow$ 数据结构变换消去 r 因子 (§7)
 上界: $O(N^{1/r})$ 存储 + $O(N^{1-1/r})$ 临时空间 + $O(r)$ 均摊时间
 $\downarrow$ 上下界匹配
 增长游戏精确解 (§8) $\rightarrow$ 时间下界 $\Omega(r)$ (§9)
 $\downarrow$
 存储-临时-时间三维 trade-off 全部紧

这条逻辑链最令我们印象深刻的不是某个单独的定理, 而是它从下界到上界、从组合游戏到数据结构、从理论到构造的**完整闭环**。上下界之间的对话贯穿始终: 乘积下界暗示了 r 参数化的形式, 而 r 参数化的上界构造反过来验证了下界的紧性。增长游戏的抽象看似偏离了原问题, 但恰恰因为去掉了数据结构实现细节, 精确分析才成为可能。

7.2 优点

1. **概念创新的力量**: 区分存储空间与临时空间这一看似简单的概念操作, 打开了一个此前被认为“已经封顶”的问题。
2. **上下界的完整匹配**: 在存储空间、临时空间和均摊时间三个维度上, 上下界全部渐近匹配, 在理论计算机科学中是理想状态。
3. **增长游戏的方法论价值**: 将数据结构操作代价建模为可精确求解的组合游戏, 这种方法论可被推广到其他问题。

4. **构造的简洁性**：基数 B 计数器式的层次化块结构出人意料地简单，再次印证了“最好的结果往往是最简单的”。
5. **参数化谱系的统一性**：将 25 年前 Brodnik 等人的单个结果 ($r = 2$) 推广为完整的一族结果，涵盖从“省空间”到“省时间”之间的所有折中点。

7.3 局限与开放问题

1. **时间下界仅适用于“标准”算法**：下界假设内存块连续分配、增长通过新块 + 拷贝实现。论文作者相信下界可推广到所有算法，但严格证明仍待完成。
2. **收缩操作的下界弱于增长**：论文对 Grow 的 $\Omega(r)$ 下界是强的，但对同时支持 Grow 和 Shrink 时的下界证明更为复杂，未给出同等强度的结果。
3. **比特复杂度的隐忧**：在 trans-dichotomous 模型下，对于较大的 r ，如何在 $O(1)$ 时间内计算块位置而不使用乘法或平方根，仍需进一步研究。
4. **实践常数因子**：第 7 节变换引入的常数因子可能偏大。在实际系统中， $r = 2$ (HAT) 或 $r = 3$ 已经足够——从工程角度看，“大量小数组、仅偶尔 resize”的场景（如多栈/多队列）适合选较大 r 以降低常驻内存；“单一大数组且频繁 grow”的场景则 $r = 2$ 或经典倍增更合适。理论的最优不等同于实践的最佳，但证明这个 trade-off 是紧的本身具有深刻的理论意义。

另外，我们认为有必要辨析论文标题中“optimal”的含义。它有三层：**渐近最优** ($O(r)$ 不能改进为 $o(r)$)；**trade-off 最优**（存储-临时-时间的三维曲面上到达 Pareto 前沿）；以及**上下界常数因子是否最优**（目前上下界的常数因子是否匹配仍是开放问题）。不同上下文中“optimal”的所指不同，阅读时需要区分。

7.4 我们的学习收获

- **对均摊分析的系统掌握**：从聚合分析、记账法到势能法，特别是势能函数在上下界证明中的对称使用——它既能证上界（分配势能支付未来昂贵操作），也能证下界（任何策略都必须积累足够势能才能支付拷贝代价）。
- **对下界证明方法论的体会**：如何从“块的数量 vs 块的大小”二分讨论出发，通过乘积形式将单一的下界强化为参数化的 trade-off 谱系。
- **对组合抽象价值的认识**：增长游戏告诉我们，一个好的抽象模型配上好的数学工具，可以把直觉变成可精确计算的公式。
- **从初读到解答的认识论收获**：我们经历了“提出疑问 → 独立推导 → 发现偏差 → 交叉验证 → 修正理解 → 代码实现 → 定性验证”的完整过程。这个过程本身比记住若干定理更有价值——它训练了我们如何深度学习一篇理论论文。

8 分工说明

组员	主要贡献
周希	初读疑问的系统提出与解答（含势能函数推导与修正、HAT 空间分解、增长游戏公式解读）；论文整体框架设计；核心算法的 Python 实现与均摊分析经验验证；对滞后设计原理、乘积下界形式、optimal 概念层次的独立思考
李宇轩	论文第 1-8 节的逐节精读与技术追踪（含后台重建全流程分析、credit 均摊账本的严格推演、第 7 节 $2r$ 参数变换的独立验算、增长游戏中 $\ell + 1 \mid N$ 整除条件的边界分析）； $r = 2$ 为何是去均摊化临界点的独立洞察；自检式阅读方法
吴贤文	论文全局架构的系统梳理与技术全景图绘制（含 $\sqrt{N}$ 下界的鲁棒性分析、变长数据块的反事实设计探究、算数/几何级数方案的对比）；第 5-10 节技术要点的完整整理；工程场景的差异化分析

参考文献

- [1] Tarjan, R. E., & Zwick, U. (2024). Optimal Resizable Arrays. *SIAM Journal on Computing*, 53(5), 1354–1380. arXiv: [2211.11009](#).
- [2] Brodnik, A., Carlsson, S., Demaine, E. D., Munro, J. I., & Sedgewick, R. (1999). Resizable Arrays in Optimal Time and Space. Technical Report CS-99-09, University of Waterloo.
- [3] Sitarski, E. (1996). HATs: Hashed Array Trees. *Dr. Dobb's Journal*, September 1996.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.), Chapter 17: Amortized Analysis. MIT Press.